



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Dubbi su Specifica Tecnica
2026-04-27

Indice

1) Informazioni generali	1
2) Domande	2
2.1) Modellazione UML _G e Convenzioni Visive	2
2.2) Scelte Architetture (Architettura Esagonale)	3
2.3) Gestione del Flusso Dati (Application Layer vs Adapter)	4
2.4) Pydantic e dominio	5
2.5) Import export di file	5
2.6) Interfacce e javascript	5

1) Informazioni generali

- **Tipo di riunione:** Esterna
- **Motivazione:** Chiarimento dubbi
- **Data:** 2026-04-27
- **Luogo:** Riunione su Zoom
- **Ora inizio:** 8.40
- **Ora fine:** 9.00
- **Partecipanti:**
 - Aldo Bettega
 - Davide Lorenzon
 - Riccardo Cardin

2) Domande

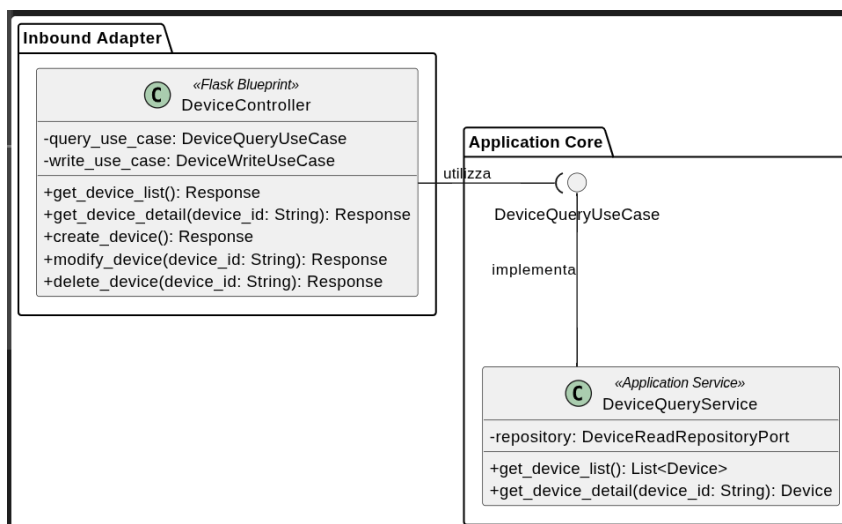
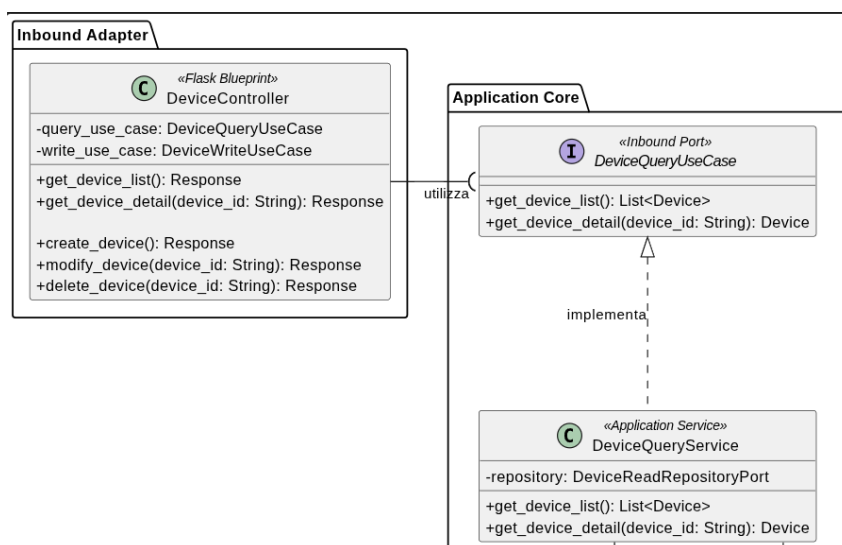
2.1) Modellazione UML_G e Convenzioni Visive

2.1.1) Rappresentazione delle Interfacce nei Diagrammi delle Classi

Per la progettazione abbiamo utilizzato PlantUML, un tool che permette di generare diagrammi uml_G tramite codice, invece di usare tool grafici.

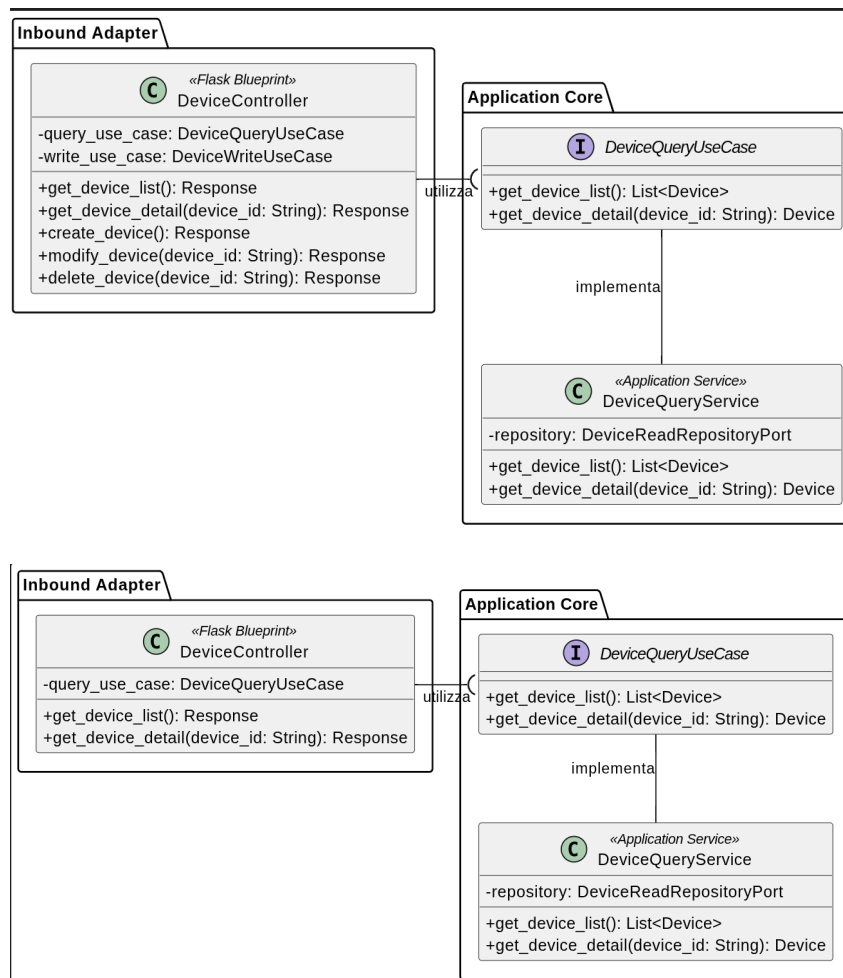
Tuttavia plant uml_G non supporta l'interfaccia UML_G 2.0 vista a lezione.

Volevamo chiedere se è accettabile usare nella specifica tecnica la sintassi UML_G 1.0 oppure una ball priva delle dichiarazioni dei metodi, eventualmente segnalati con apposita nota



2.1.2) Livello di Dettaglio degli Adapter nei Diagrammi Parziali

Quando un singolo Adapter implementa molteplici Porte, come deve essere rappresentato nei diagrammi delle classi settoriali (es. diagrammi focalizzati su un singolo caso d'uso)? Deve mostrare l'elenco completo di tutti i metodi che possiede, oppure è preferibile omettere i metodi non rilevanti per il contesto specifico?



2.2) Scelte Architetture (Architettura Esagonale)

2.2.1) Molteplicità delle Porte per un Singolo Adapter

Dal punto di vista del design architeturale (Porte e Adattatori), è considerata una buona pratica definire molteplici Outbound Ports (segregate per logica o entità) che vengono poi implementate a livello infrastrutturale da un unico Adapter concreto (es. un unico MongoAdapter che implementa sia DeviceRepositoryPort che StandardRepositoryPort)?

2.2.2) Separazione delle Responsabilità (Query vs Command)

È corretto e consigliabile separare a livello di Use Case e Service le operazioni di lettura (Read/Query) da quelle di modifica/scrittura (Write/Command), seguendo un approccio ispirato al CQRS (Command Query Responsibility Segregation)?

Un'applicazione estremamente rigorosa del Single Responsibility Principle porterebbe alla definizione di interfacce funzionali, volevamo sapere se applicare questa separazione oppure i raggruppamenti prima descritti sono accettabili

2.3) Gestione del Flusso Dati (Application Layer vs Adapter)

2.3.1) Restituzione dei Dati al Controller: DTO vs Entità di Dominio

L'applicazione opera con un'interfaccia_G web, l'utilizzo di DTO granulari per gli utilizzi (EntitySummaryDTO e EntityDetailDTO) è fondamentale per la comunicazione tra backend e frontend.

Il nostro dubbio riguarda dove definire e far vivere i DTO, le opzioni individuate sono le seguenti:

- Le porte ritornano dei DTO costruiti dai service

Vantaggi

- Le classi di Dominio non vengono rese note
- Maggiore decoupling

Svantaggi

- Appesantisce il Dominio con codice boilerplate
- L'uso di Pydantic renderebbe più veloce l'implementazione, ma introduce una dipendenza_G esterna all'interno della business logic

- Le porte ritornano degli oggetti di dominio

Vantaggi

- Domain leggero

Svantaggi

- Viene esposto un oggetto di Dominio, teoricamente corretto, ma permette di modificare l'oggetto dall'esterno

Possibili mitigazioni

- Invece di un oggetto di dominio si restituisce una classe view che realizza un wrapper che nasconde i metodi che modifica lo stato interno.

In linguaggio di programmazione tipizzato staticamente sarebbe l'equivalente di un `const`

Per alleggerire le firme di funzione si stava valutando di usare DTO invece di un elenco di parametri.

Anche se questa pratica è lecita va valutato il trade off tra codice boilerplate per il DTO di modifica e l'effettivo risparmio di codice ed estensibilità

2.4) Pydantic e dominio

Pydantic offre una sintassi molto snella per la definizione di regole di verifica per la validazione semantica dei dati.

Se si optasse per utilizzare DTO nel dominio l'uso di pydantic per la loro definizione tornerebbe utile a ridurre la verbosità.

2.5) Import export di file

In generale si preferisce mantenere i dati come raw byte o classi della libreria standard che permettono di rappresentare i file oppure si utilizzano dei DTO di dominio

2.6) Interfacce e javascript

Javascript non supporta le interfacce.

È accettabile dichiarare un'interfaccia nel diagramma UML per dire che si può inserire una qualsiasi classe che implementa i metodi dichiarati dall'interfaccia